


BASES DE DATOS 2 · INGENIERÍA DE SISTEMAS

# Bases de Datos Post-relacionales: NoSQL

*Qué problemas resuelven, cómo modelan los datos, qué resignan a cambio — y cómo se trabaja con MongoDB.*

---

Motivación · CAP, ACID y BASE · Los 4 modelos de datos · NoSQL vs relacional · MongoDB: modelado y operaciones

# ¿Qué vamos a poder hacer al final de esta clase?

1. **Explicar por qué existen las bases NoSQL**
   Qué problemas de escala y flexibilidad resuelven, y qué garantías resignan a cambio (CAP, ACID vs BASE).

2. **Distinguir los cuatro modelos de datos y elegir entre ellos**
   Documentos, clave-valor, columna ancha y grafos: cómo guardan un mismo dato y para qué consulta sirve cada uno.

3. **Modelar y operar una base documental con MongoDB**
   Embeber vs referenciar, CRUD con operadores, agregaciones, índices y lectura de planes con explain().

INTRODUCCIÓN

# Recorrido de la clase

1

## ¿Por qué NoSQL?

Escalar, esquema flexible, replicación y sharding

2

## La teoría

Teorema CAP, ACID vs BASE, consistencia eventual

3

## Modelos de datos

Documentos, clave-valor, columna ancha, grafos

4

## NoSQL vs relacional

Comparación honesta, mitos, persistencia políglota

5

## MongoDB

Vocabulario, modelado, replicación, sharding, transacciones

6

## Operaciones

CRUD, operadores, agregación, índices y explain()

# El problema que dio origen a NoSQL

A mediados de los 2000, empresas como Google y Amazon tenían aplicaciones con cientos de millones de usuarios. Un único servidor relacional —por grande que fuera— ya no alcanzaba.

**VOLUMEN**

Terabytes y petabytes de datos: logs, clics, mensajes, sensores.

**VARIEDAD**

Datos con estructura cambiante o semi-estructurados (JSON, eventos).

**VELOCIDAD**

Miles de escrituras por segundo, respuestas en milisegundos.

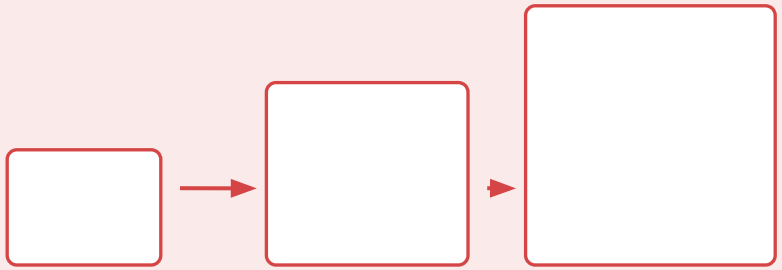
**DISPONIBILIDAD**

El servicio no puede caerse aunque falle un servidor o un datacenter.

**NoSQL = "Not Only SQL".** No reemplaza al modelo relacional: agrega otras formas de guardar datos para problemas donde lo relacional no es la mejor herramienta.

# Escalar: agrandar la máquina o sumar máquinas

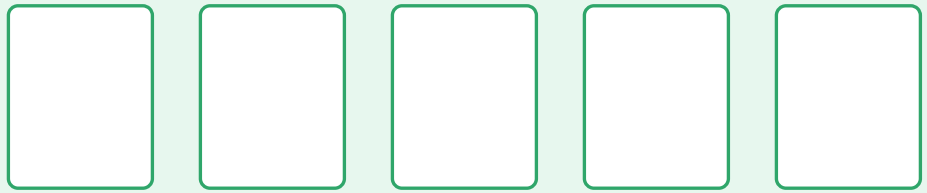
### ESCALADO VERTICAL (scale up)



**Más CPU, RAM y disco en un solo servidor.**

- Simple: la aplicación no cambia.
- Tiene un techo físico y el costo crece más rápido que la capacidad.
- Sigue siendo un único punto de falla.

### ESCALADO HORIZONTAL (scale out)



**Más servidores (nodos) comunes que se reparten datos y carga.**

- Casi sin techo: se agregan nodos a medida que crece.
- Hardware barato; si un nodo cae, los demás siguen.
- Costo: el motor tiene que coordinar datos distribuidos.

*Los motores relacionales también pueden escalar horizontalmente, pero suele ser más complejo. Los NoSQL nacieron diseñados para hacerlo.*

# Esquema fijo vs esquema flexible

### RELACIONAL · schema-on-write

La estructura se define antes de guardar. Todo lo que no respete la tabla se rechaza.

SQL

```sql
CREATE TABLE usuario (
  id       INT PRIMARY KEY,
  nombre   VARCHAR(60) NOT NULL,
  email    VARCHAR(100),
  ciudad   VARCHAR(40)
);
-- ¿Un usuario con 2 teléfonos?
-- → ALTER TABLE o una tabla nueva
```

### NOSQL DOCUMENTAL · schema-on-read

Cada documento trae su propia estructura. La aplicación la interpreta al leer.

misma colección

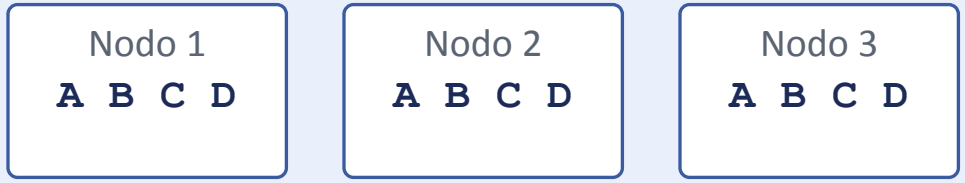
```json
{ "_id": 1, "nombre": "Ana",
  "email": "ana@mail.com" }

{ "_id": 2, "nombre": "Luis",
  "telefonos": ["249-111", "249-222"],
  "direccion": { "ciudad": "Tandil" } }
```

*"Sin esquema" no significa "sin reglas": el esquema vive en la aplicación (y se puede validar en el motor).*

# Replicación y sharding: dos técnicas que no hay que confundir

## REPLICACIÓN → disponibilidad




**Copia completa en cada nodo.**

- Si un nodo falla, otro sigue respondiendo.
- Permite repartir las lecturas.
- No aumenta la capacidad total de almacenamiento.

## SHARDING → escalabilidad

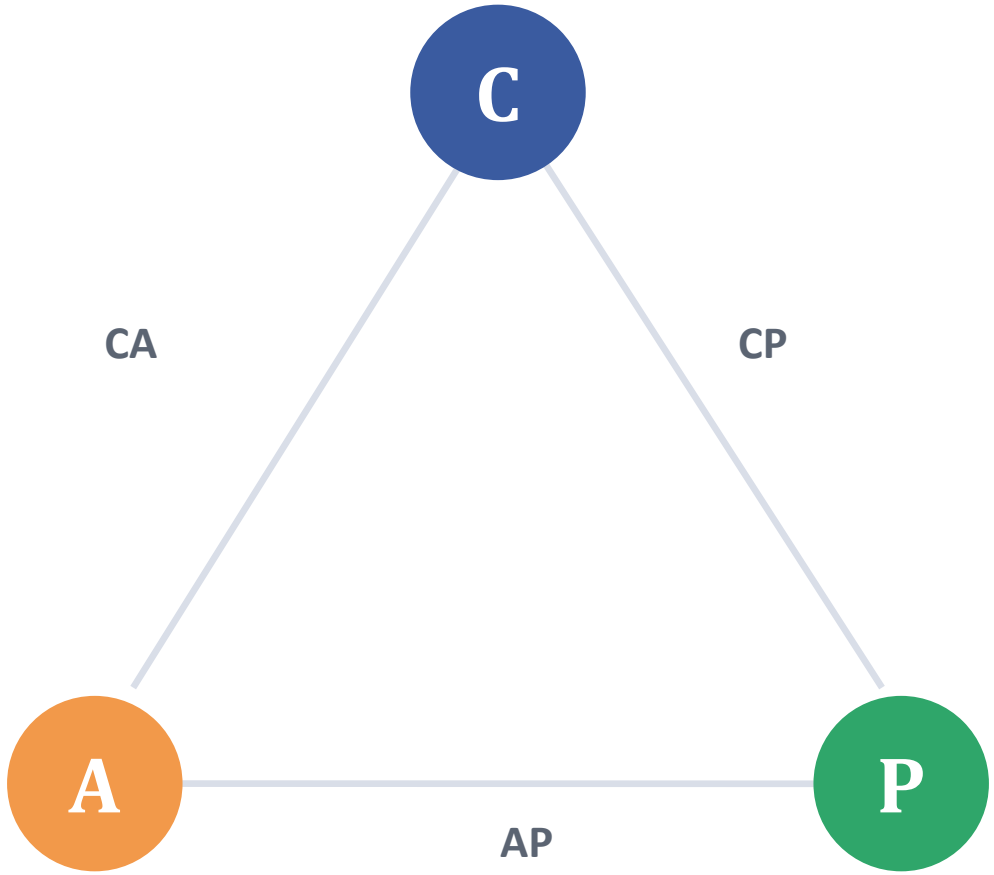



**Cada nodo guarda solo una parte.**

- Reparte volumen y escrituras entre nodos.
- Una clave de partición decide a qué shard va cada dato.
- Por sí solo no da tolerancia a fallos.

*En la práctica se combinan: los datos se parten en shards y cada shard se replica.*

# El teorema CAP




Un sistema distribuido no puede garantizar las tres propiedades a la vez cuando la red falla:

**C** **Consistencia**

Toda lectura devuelve el último valor escrito, sin importar a qué nodo se consulte.

**A** **Disponibilidad**

Toda petición a un nodo que funciona recibe una respuesta (aunque no sea la más reciente).

**P** **Tolerancia a particiones**

El sistema sigue operando aunque se corte la comunicación entre nodos.

*Ojo: la "C" de CAP (todos los nodos ven el mismo dato) no es la "C" de ACID (reglas de integridad).*

# Ante una partición: ¿consistencia o disponibilidad?

En un sistema distribuido las fallas de red van a ocurrir, así que la P no es opcional. La decisión real es qué hacer cuando la red se corta:

### CP — PRIORIZA CONSISTENCIA

Si no puede garantizar el dato correcto, prefiere no responder (o rechazar la escritura).

Ejemplos: MongoDB (configuración por defecto), HBase.

Uso típico: saldos, stock, reservas.

### AP — PRIORIZA DISPONIBILIDAD

Siempre responde, aunque algún nodo devuelva un dato desactualizado por un momento.

Ejemplos: Cassandra, CouchDB, DynamoDB (lecturas eventuales).

Uso típico: likes, feeds, carritos, catálogos.

**Muchos motores permiten ajustar esta elección por operación** (tunable consistency): por ejemplo, niveles de consistencia en Cassandra o write/read concern en MongoDB.

# ACID vs BASE: dos filosofías de garantías

## ACID · MODELO RELACIONAL CLÁSICO

**A — Atomicidad**
La transacción se hace completa o no se hace.

**C — Consistencia**
Pasa de un estado válido a otro válido.

**I — Aislamiento**
Las transacciones concurrentes no se pisan.

**D — Durabilidad**
Lo confirmado no se pierde.

## BASE · MUCHOS SISTEMAS NOSQL

**BA — Basically Available**
El sistema responde casi siempre, aunque haya fallas parciales.

**S — Soft state**
El estado puede cambiar solo mientras las réplicas se sincronizan.

**E — Eventual consistency**
Si no hay nuevas escrituras, todas las réplicas terminan con el mismo valor.

*ACID prioriza la corrección; BASE, la disponibilidad y la escala. Hoy muchos motores ofrecen ambas.*

# Consistencia eventual, con un ejemplo cotidiano

Le das "me gusta" a una publicación. El sistema tiene tres réplicas en distintas regiones:




**Tolera la demora:** contadores, feeds, comentarios, catálogos, recomendaciones.

**No la tolera:** saldo de una cuenta, stock de la última unidad, asientos de un vuelo.

# Los cuatro modelos de datos NoSQL

### Documentos

Cada registro es un documento JSON/BSON autocontenido, con anidamiento.

*MongoDB · Couchbase · Firestore*

### Clave-valor

Un diccionario gigante: una clave única apunta a un valor opaco.

*Redis · DynamoDB · Memcached*

### Columna ancha

Filas agrupadas por clave de partición, con columnas que pueden variar entre filas.

*Cassandra · HBase · Bigtable*

### Grafos

Nodos y relaciones como ciudadanos de primera clase, ambos con propiedades.

*Neo4j · Amazon Neptune · ArangoDB*

# Documentos: el dato como una unidad autocontenida

## ¿Qué es?

Cada registro es un documento (JSON, BSON o XML) que puede contener objetos y arrays anidados. Los documentos se agrupan en colecciones.

## Características

- Estructura flexible: cada documento puede tener campos distintos.
- Lo que se lee junto se guarda junto (desnormalización deliberada).
- Se consulta por cualquier campo, incluso anidado, y admite índices secundarios.
- Escala horizontalmente con sharding y replica para alta disponibilidad.

colección: usuarios

```
{
  "_id": 1,
  "nombre": "Ana Pérez",
  "email": "ana@mail.com",
  "direccion": {
    "calle": "Pinto 399",
    "ciudad": "Tandil"
  },
  "intereses": ["running", "tecnología"],
  "pedidos": [
    { "id": 501, "total": 880000 },
    { "id": 502, "total": 15000 }
  ]
}
```

# Documentos: cuándo conviene y principales motores

**CONVIENE CUANDO…**

- Las entidades tienen estructura variable (catálogo de productos con atributos distintos).
- Se lee y escribe el agregado completo (un pedido con sus ítems).
- El esquema evoluciona rápido.

**NO CONVIENE CUANDO…**

- Hay muchas relaciones muchos-a-muchos que se consultan cruzadas.
- Se necesitan reportes ad-hoc con muchos JOINs.

## Principales motores

**MongoDB**

El más adoptado. BSON, lenguaje de consultas y agregación propio, servicio gestionado Atlas.

**Couchbase**

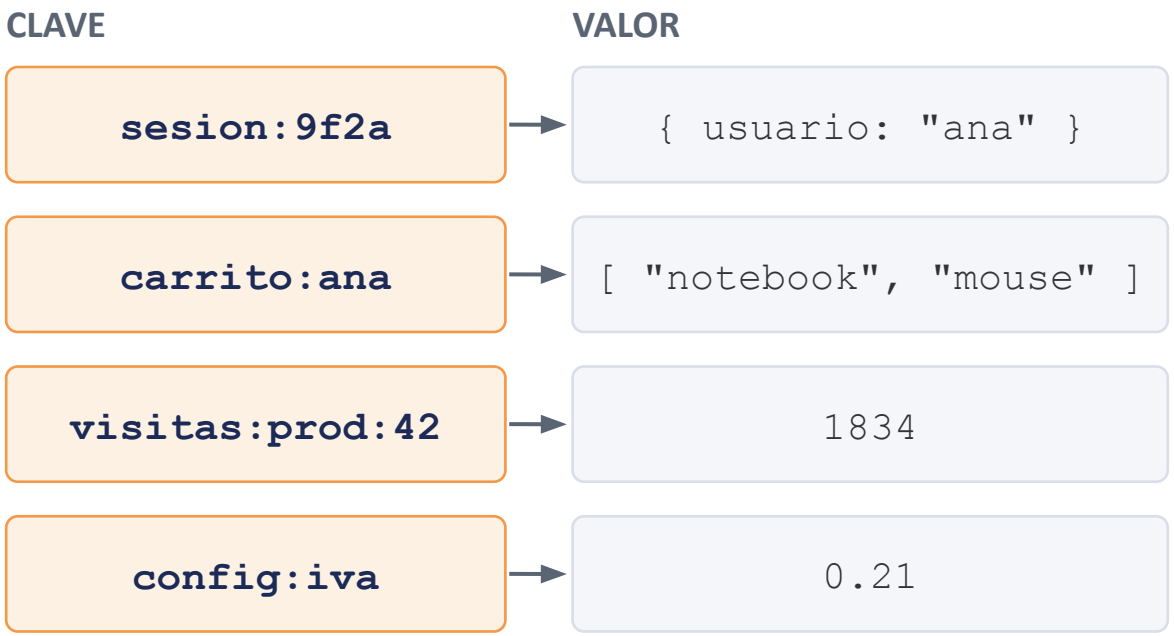
Caché en memoria + persistencia. Consultas con SQL++ (ex N1QL). Sync para móviles.

**Firebase Firestore**

De Google. Pensado para apps web/móviles: sincronización en tiempo real y modo offline.

# Clave-valor: un diccionario distribuido

Cada dato se guarda como un par clave → valor. El motor solo sabe buscar por clave, y por eso es extremadamente rápido (en general, en memoria).




redis-cli

```
# guardar la sesión por 1 hora
SET sesion:9f2a "ana" EX 3600

# leerla
GET sesion:9f2a

# contador atómico
INCR visitas:prod:42

# lista: agregar al carrito
RPUSH carrito:ana "notebook" "mouse"
```

# Clave-valor: cuándo conviene y principales motores

**CONVIENE CUANDO…**

- Siempre se accede por un identificador conocido.
- Se necesita latencia de milisegundos o menos.
- Los datos son temporales: sesiones, caché, tokens, colas.

**NO CONVIENE CUANDO…**

- Hay que buscar por atributos del valor ("usuarios de Tandil").
- Los datos tienen relaciones o requieren consultas complejas.

## Principales motores

R

### Redis

En memoria con persistencia opcional. Estructuras avanzadas: listas, sets, sorted sets, hashes. Caché y colas.

D

### Amazon DynamoDB

Servicio gestionado, clave-valor y documental. Consistencia eventual o fuerte, a elección por lectura.

M

### Memcached

Solo caché en memoria, sin persistencia. Más simple que Redis, muy eficiente para ese único fin.

# Ojo: "orientada a columnas" nombra dos cosas distintas

### COLUMNA ANCHA (WIDE-COLUMN) · NoSQL

- Filas agrupadas por una clave de partición, repartidas entre nodos.
- Cada fila puede tener columnas distintas.
- Optimizada para escrituras masivas y lecturas por clave (+ rango).
- Uso: mensajería, IoT, series temporales, historial de actividad.

**Cassandra · HBase · Google Bigtable · ScyllaDB**

### ALMACENAMIENTO COLUMNAR · OLAP

- Guarda en disco cada columna por separado.
- Optimizado para leer pocas columnas de millones de filas.
- Comprime muy bien; las escrituras fila a fila son más costosas.
- Uso: data warehouse, reportes, analítica. Se consultan con SQL.

**Amazon Redshift · Google BigQuery · ClickHouse · Vertica**

*Los dos comparten el nombre pero resuelven problemas casi opuestos: uno está pensado para escribir mucho y rápido; el otro, para analizar mucho y rápido.*

# Almacenamiento columnar: por qué acelera las agregaciones

**POR FILAS (row-store)**

| | | | |
|---|---|---|---|
| 1 | Ana | Tandil | 500 |
| 2 | Luis | Rosario | 300 |
| 3 | Eva | Tandil | 200 |

Bloques en disco = filas completas. Para SUM(monto) hay que leer los 3 bloques enteros.

**POR COLUMNAS (column-store)**

| | | | |
|---|---|---|---|
| 1 | 2 | 3 | *id* |
| Ana | Luis | Eva | *nombre* |
| Tandil | Rosario | Tandil | *ciudad* |
| 500 | 300 | 200 | *monto* |

Bloques en disco = columnas. Para SUM(monto) se lee solo el bloque de monto.

```
SELECT ciudad, SUM(monto) FROM ventas GROUP BY ciudad;
-- lee 2 de 4 columnas: en una tabla con 100 columnas y
-- 1.000 millones de filas, la diferencia es enorme
```

# Columna ancha con Cassandra: diseñar la tabla para la consulta

## Consulta que necesitamos: "los últimos pedidos de un usuario"

cqlsh (CQL)

```
CREATE TABLE pedidos_por_usuario (
  usuario_id  uuid,
  fecha       timestamp,
  pedido_id   uuid,
  total       decimal,
  PRIMARY KEY ((usuario_id), fecha, pedido_id)
) WITH CLUSTERING ORDER BY
  (fecha DESC, pedido_id ASC);

SELECT * FROM pedidos_por_usuario
 WHERE usuario_id = ? LIMIT 10;
```

P

### Clave de partición

usuario_id: todos los pedidos de un usuario quedan juntos en el mismo nodo.

K

### Clave de clustering

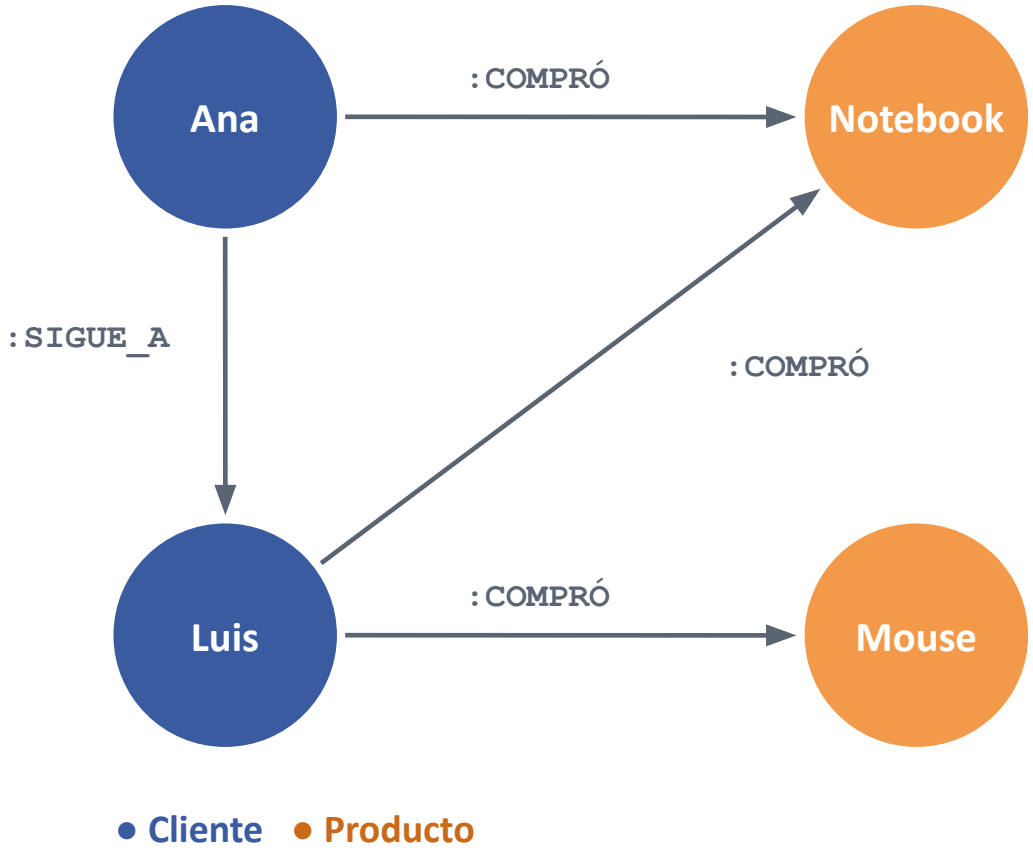
fecha DESC: dentro de la partición quedan ya ordenados, del más nuevo al más viejo.

!

### Sin JOINs

¿Otra consulta (pedidos por producto)? → otra tabla, con los datos duplicados.

# Grafos: cuando lo importante son las relaciones




## ¿Qué es?

Los datos se modelan como nodos (entidades) y aristas (relaciones). Ambos pueden tener propiedades, por ejemplo  COMPRÓ {fecha, cantidad}.

## Características

- Las relaciones se guardan, no se calculan: recorrerlas no requiere JOINs.
- El costo depende de la parte del grafo que se recorre, no del tamaño total.
- Lenguajes propios: Cypher, Gremlin, SPARQL.
- Algoritmos integrados: camino más corto, centralidad, comunidades.

# Grafos: una recomendación en Cypher (y su equivalente en SQL)

## "Productos que compraron otros clientes que compraron lo mismo que Ana"

Neo4j · Cypher

```
MATCH (ana:Cliente {nombre:'Ana'})
      -[:COMPRO]->(p:Producto)
      <-[:COMPRO]-(otro:Cliente)
      -[:COMPRO]->(rec:Producto)
WHERE NOT EXISTS {
  (ana)-[:COMPRO]->(rec) }
RETURN rec.nombre, count(*) AS veces
ORDER BY veces DESC LIMIT 5;
```

SQL equivalente

```
SELECT r.nombre, COUNT(*) AS veces
FROM compra a
 JOIN compra o ON o.producto_id = a.producto_id
              AND o.cliente_id <> a.cliente_id
 JOIN compra c ON c.cliente_id = o.cliente_id
 JOIN producto r ON r.id = c.producto_id
WHERE a.cliente_id = :ana
  AND c.producto_id NOT IN (SELECT producto_id
       FROM compra WHERE cliente_id = :ana)
GROUP BY r.nombre
ORDER BY veces DESC LIMIT 5;
```

**Motores:** Neo4j (Cypher) · Amazon Neptune (Gremlin, openCypher, SPARQL) · ArangoDB (multimodelo)

# Los cuatro modelos, lado a lado

| | Documentos | Clave-valor | Columna ancha | Grafos |
|---|---|---|---|---|
| **Unidad de dato** | Documento JSON/BSON | Par clave ⟶ valor | Fila dentro de una partición | Nodo y relación |
| **Se consulta por** | Cualquier campo (con índices) | Solo la clave | Clave de partición (+ rango) | Patrones y recorridos |
| **Punto fuerte** | Flexibilidad, agregados completos | Velocidad extrema | Escrituras masivas, escala | Relaciones profundas |
| **Punto débil** | Relaciones muchos-a-muchos | No busca por contenido | Consultas no previstas | Agregaciones masivas |
| **Casos típicos** | Catálogos, perfiles, CMS | Caché, sesiones, colas | IoT, logs, mensajería | Redes sociales, fraude, recomendaciones |
| **Motores** | MongoDB, Couchbase | Redis, DynamoDB | Cassandra, HBase | Neo4j, Neptune |

# Una comparación honesta: casi todo "depende"

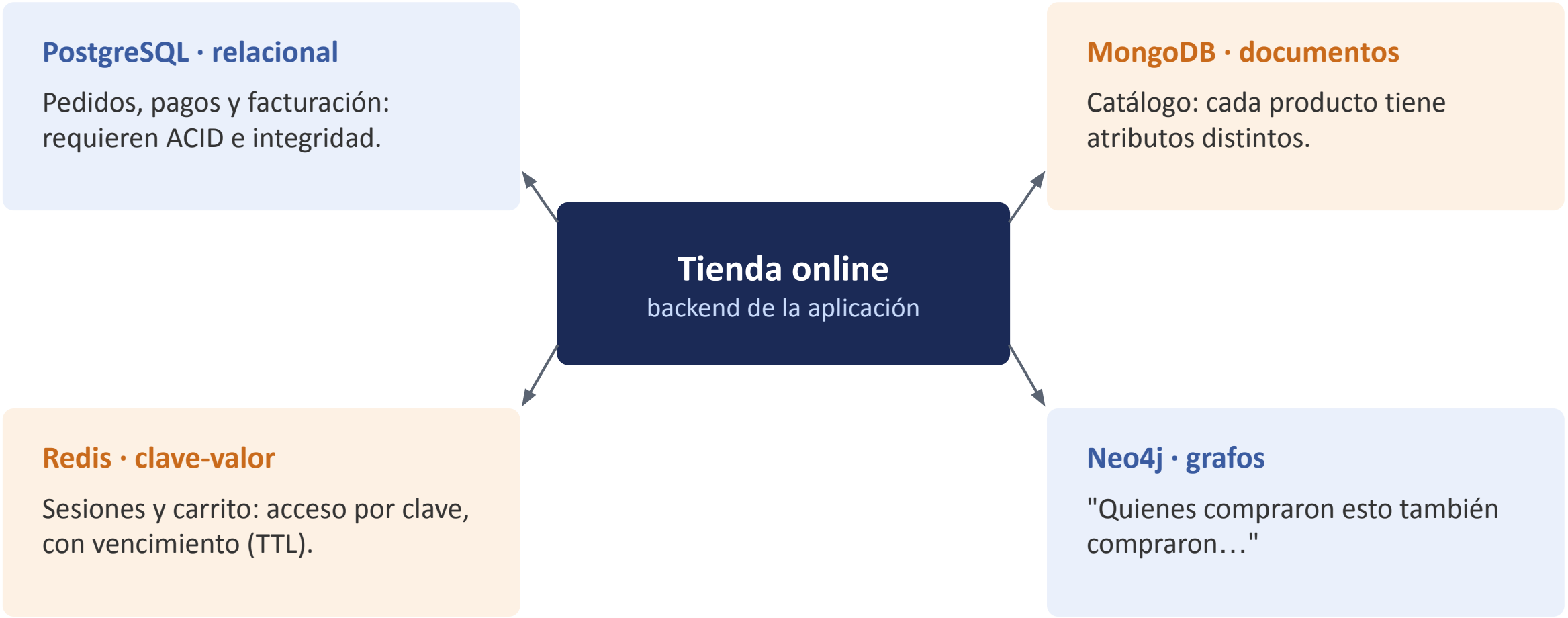
| Aspecto | Relacional | NoSQL (en general) |
|---|---|---|
| **Modelo** | Tablas normalizadas y relaciones por claves | Documento, clave-valor, columna ancha o grafo |
| **Esquema** | Fijo, definido antes de guardar (schema-on-write) | Flexible, se interpreta al leer (schema-on-read) |
| **Escalado típico** | Vertical; el horizontal es posible pero más complejo | Horizontal por diseño (sharding + replicación) |
| **Consistencia** | Fuerte, transacciones ACID | Configurable: de eventual a fuerte según motor y operación |
| **Consultas** | SQL estándar, JOINs y consultas ad-hoc | API o lenguaje propio; rinde mejor en consultas previstas |
| **Rendimiento** | Muy bueno con buen modelado e índices | Muy bueno para su patrón de acceso; malo fuera de él |
| **Mejor para** | Datos muy relacionados, integridad, reportes | Gran escala, estructura variable, acceso por clave conocido |

*Regla práctica: el relacional sigue siendo la opción por defecto; NoSQL se elige cuando hay una razón concreta.*

# Cinco mitos frecuentes sobre NoSQL

| | |
|---|---|
| ✗ "NoSQL es más rápido que SQL" | Depende de la consulta. Para leer por clave suele serlo; para un reporte con JOINs puede ser mucho más lento. |
| ✗ "NoSQL no tiene transacciones" | MongoDB, Neo4j y DynamoDB ofrecen transacciones ACID. |
| ✗ "NoSQL no tiene esquema" | El esquema existe: vive en la aplicación y también puede validarse en el motor. |
| ✗ "NoSQL no usa SQL" | Es "Not Only SQL": Cassandra usa CQL, Couchbase SQL++, y muchos motores aceptan dialectos SQL. |
| ✗ "NoSQL vino a reemplazar a SQL" | La mayoría de los sistemas reales combinan ambos (persistencia políglota). |

# Persistencia políglota: cada dato en el motor que mejor le calza




*La pregunta de diseño no es "¿SQL o NoSQL?", sino "¿qué motor para qué parte del sistema?".*

# ¿Cuál elijo? Cinco preguntas guía

| | |
|---|---|
| ¿Necesitás integridad fuerte, JOINs y consultas ad-hoc? | → **Relacional** |
| ¿Accedés siempre por un identificador y necesitás velocidad extrema? | → **Clave-valor** |
| ¿Tus entidades son agregados autocontenidos con estructura variable? | → **Documentos** |
| ¿Escribís muchísimo y consultás por una clave + rango de tiempo? | → **Columna ancha** |
| ¿Lo central son las relaciones y recorrerlas a varios niveles? | → **Grafos** |

*Si ninguna respuesta es un "sí" claro, el relacional suele ser la opción más segura.*

# MongoDB en una diapositiva

### { } Documentos BSON

JSON binario: agrega tipos como fechas, enteros de 64 bits, decimales y ObjectId.

### _id Clave primaria

Todo documento tiene un _id único; si no se indica, se genera un ObjectId.

### C Colecciones

Agrupan documentos; se crean solas con el primer insert.

### RS Replica sets

Alta disponibilidad con elección automática de un nuevo primario.

### SH Sharding

Escalado horizontal repartiendo la colección entre shards.

### >_ Herramientas

mongosh (consola), Compass (GUI), drivers para todos los lenguajes, Atlas (nube).

# Vocabulario: de SQL a MongoDB

| SQL | MongoDB |
|---|---|
| Base de datos | Base de datos |
| Tabla | Colección |
| Fila (registro) | Documento |
| Columna | Campo |
| Clave primaria | _id |
| JOIN | Documento embebido o $lookup |
| GROUP BY | Pipeline de agregación ($group) |
| Índice | Índice |

un documento de la colección usuarios

```
{
  "_id": ObjectId("66f7c2..."),
  "nombre": "Juan",
  "edad": 25,
  "direccion": {
    "ciudad": "Rosario",
    "cp": "2000"
  },
  "aficiones": [
    { "nombre": "Fútbol" },
    { "nombre": "Ajedrez" }
  ]
}
```

# Esquema flexible… pero con validación

Dos documentos distintos pueden convivir en la misma colección. Cuando conviene poner reglas, MongoDB permite validar con JSON Schema:

mongosh

```
db.createCollection("usuarios", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nombre", "email"],
      properties: {
        email: { bsonType: "string", pattern: "@" },
        edad:  { bsonType: "number", minimum: 0 }
      }
    }
  }
});
```

**SE ACEPTA**

{ nombre: "Ana",
email: "ana@mail.com",
telefono: "249-111" }

**SE RECHAZA**

{ nombre: "Luis", edad: -3 }
→ falta email y edad < 0
(Document failed validation)

# La decisión de modelado clave: embeber o referenciar

### EMBEBER (documento dentro de documento)

```
{ _id: 1, nombre: "Ana",
  direcciones: [
    { calle: "Pinto 399",
      ciudad: "Tandil" }
  ] }
```

- Los datos se leen juntos casi siempre.
- Relación uno-a-pocos (y acotada).
- El dato hijo no tiene sentido por separado.
- Una lectura, sin JOIN; se actualiza en forma atómica.

### REFERENCIAR (guardar el _id del otro)

```
// usuarios
{ _id: 1, nombre: "Ana" }
// pedidos
{ _id: 501, usuario_id: 1,
  total: 880000 }
```

- Relación uno-a-muchos que crece sin límite.
- El dato se comparte entre muchos documentos.
- Se consulta o actualiza por separado.
- Se une con $lookup o desde la aplicación.

*Límite técnico: un documento BSON no puede superar los 16 MB.*

# Regla de oro: modelá según cómo vas a consultar

## RELACIONAL · normalizado




**Mostrar un pedido = 3 tablas y 2 JOINs.**

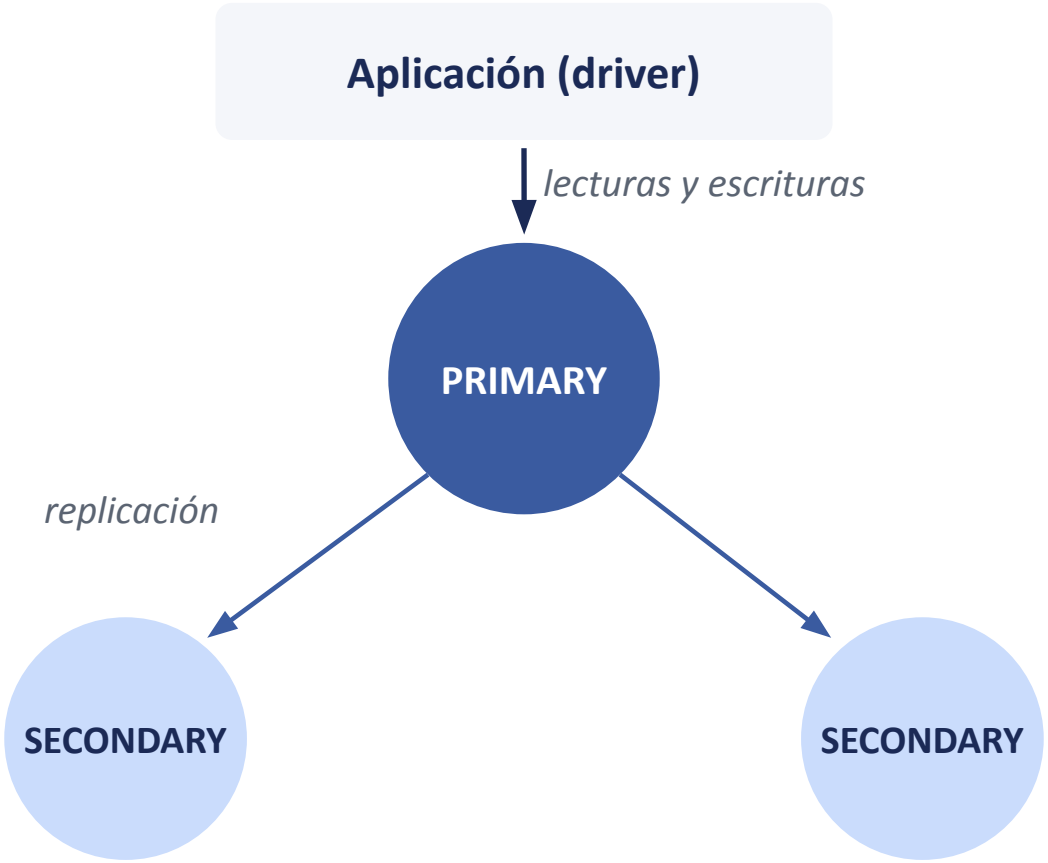
## DOCUMENTAL · un agregado por pedido

colección: pedidos

```
{
  _id: 501,
  fecha: ISODate("2026-09-28"),
  cliente: { id: 1, nombre: "Ana" },
  items: [
    { prod: "Notebook", precio: 850000, cant: 1 },
    { prod: "Mouse",    precio:  15000, cant: 2 }
  ],
  total: 880000
}
```

*Copiar nombre y precio del producto dentro del pedido es desnormalización deliberada: además conserva el precio al momento de la compra.*

# Alta disponibilidad: Replica Set



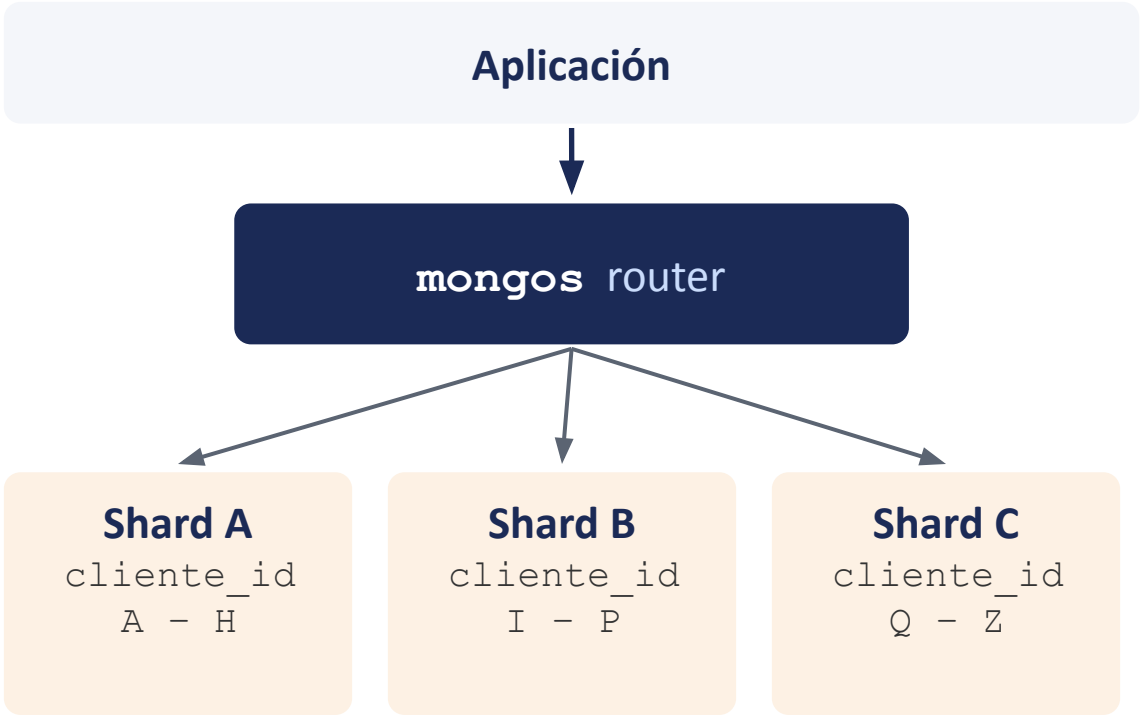

1. **Un único primario**
   Recibe todas las escrituras y las registra en su oplog.

2. **Secundarios que copian**
   Aplican el oplog del primario y mantienen el mismo conjunto de datos.

3. **Elección automática**
   Si el primario cae, los nodos votan y un secundario pasa a ser primario.

4. **Número impar de nodos**
   Se recomiendan 3 o más, para que siempre pueda formarse una mayoría.

# Escalado horizontal: sharding y la clave de partición




*cada shard es un replica set*

## ¿Cómo se reparte?

Se elige una shard key; el router (mongos) usa esa clave para mandar cada documento y cada consulta al shard que corresponde.

**BUENA SHARD KEY**

Muchos valores distintos, bien repartidos, y presente en las consultas frecuentes.

**MALA SHARD KEY**

Una fecha o un valor que siempre crece: todas las escrituras nuevas caen en el mismo shard (hotspot).

# Transacciones en MongoDB

### SIEMPRE: UN DOCUMENTO ES ATÓMICO

Cambiar varios campos de un mismo documento es todo-o-nada.

### DESDE 4.0: MULTI-DOCUMENTO

ACID entre documentos y colecciones: 4.0 en replica sets, 4.2 con sharding.

**Consejo de diseño:** si todo necesita transacciones multi-documento, revisá el modelo (¿embeber?, ¿relacional?).

transferencia entre cuentas

```
const s = db.getMongo().startSession();
const c = s.getDatabase("banco").cuentas;

s.startTransaction();
try {
  c.updateOne({ _id: "A" },
              { $inc: { saldo: -100 } });
  c.updateOne({ _id: "B" },
              { $inc: { saldo:  100 } });
  s.commitTransaction();
} catch (e) {
  s.abortTransaction();  // se deshacen ambas
}
```

# Insertar documentos

```
db.<coleccion>.insertOne({ ... })
db.<coleccion>.insertMany([ {...}, {...} ])
```

mongosh

```
db.usuarios.insertOne({
  nombre: "Juan", edad: 30, ciudad: "Tandil",
  email: "juan@mail.com", intereses: ["futbol"]
});

db.usuarios.insertMany([
  { nombre: "Ana",    edad: 25, ciudad: "Rosario" },
  { nombre: "Carlos", edad: 40, ciudad: "Córdoba",
    direccion: { calle: "Colón 1200", cp: "5000" } }
]);
```

**PARA NOTAR**

- Si la colección no existe, se crea.
- Si no se indica _id, MongoDB genera un ObjectId único.
- Los documentos de un mismo insertMany pueden tener campos distintos.
- Responde con los _id insertados.

# Consultar documentos: filtro, proyección, orden y límite

```
db.<coleccion>.find( { FILTRO }, { PROYECCIÓN } ).sort({...}).limit(n)
```

| MongoDB | SQL equivalente |
|---|---|
| `db.usuarios.find()` | `SELECT * FROM usuarios` |
| `db.usuarios.find({ ciudad: "Tandil" })` | `… WHERE ciudad = 'Tandil'` |
| `db.usuarios.find({ ciudad: "Tandil" }, { nombre: 1, edad: 1, _id: 0 })` | `SELECT nombre, edad … WHERE ciudad = 'Tandil'` |
| `db.usuarios.find().sort({ edad: -1 }).limit(5)` | `… ORDER BY edad DESC LIMIT 5` |
| `db.usuarios.countDocuments( { ciudad: "Tandil" })` | `SELECT COUNT(*) … WHERE ciudad = 'Tandil'` |

# Operadores de consulta

| Qué querés | MongoDB | SQL aproximado |
|---|---|---|
| **Comparar** | `{ edad: { $gte: 18, $lt: 30 } }` | `edad >= 18 AND edad < 30` |
| **Distinto** | `{ ciudad: { $ne: "Tandil" } }` | `ciudad <> 'Tandil'` |
| **Uno de varios** | `{ ciudad: { $in: ["Tandil", "Azul"] } }` | `ciudad IN ('Tandil','Azul')` |
| **O lógico** | `{ $or: [ { edad: { $lt: 18 } }, { ciudad: "Azul" } ] }` | `edad < 18 OR ciudad = 'Azul'` |
| **Campo anidado** | `{ "direccion.cp": "5000" }` | `(requiere JOIN con dirección)` |
| **Dentro de un array** | `{ intereses: "futbol" }` | `(requiere tabla intermedia)` |
| **Existe el campo** | `{ email: { $exists: false } }` | `email IS NULL (aprox.)` |

*Varias condiciones en el mismo objeto se combinan con AND implícito: { ciudad: "Tandil", edad: { $gt: 30 } }*

# Actualizar documentos

```
db.<coleccion>.updateOne ( { FILTRO }, { $set: {...} } )   // el primero
db.<coleccion>.updateMany( { FILTRO }, { $set: {...} } )   // todos
```

mongosh

```
// un usuario concreto (campo único)
db.usuarios.updateOne(
  { email: "juan@mail.com" },
  { $set: { edad: 31 } });

// todos los de una ciudad
db.usuarios.updateMany(
  { ciudad: "Tandil" },
  { $set: { region: "Centro" } });
```

| Operador | Efecto |
|---|---|
| `$set` | Asigna o agrega un campo |
| `$unset` | Elimina un campo |
| `$inc` | Suma (o resta) un valor |
| `$push` | Agrega a un array |
| `$pull` | Quita de un array |

# Actualizar: varios operadores y upsert

mongosh

```
// varios operadores a la vez
db.usuarios.updateOne(
  { nombre: "Ana" },
  { $inc:  { compras: 1 },
    $push: { intereses: "running" } });

// si no existe, lo crea (upsert)
db.usuarios.updateOne(
  { email: "leo@mail.com" },
  { $set: { nombre: "Leo" } },
  { upsert: true });
```

**UPSERT = UPDATE + INSERT**

Si ningún documento coincide con el filtro, crea uno nuevo con el filtro y los cambios.

**QUÉ DEVUELVE**

matchedCount: cuántos coincidieron

modifiedCount: cuántos cambiaron

upsertedId: _id del creado (si hubo)

# Eliminar documentos

```
db.<coleccion>.deleteOne ( { FILTRO } )   // el primero que coincide
db.<coleccion>.deleteMany( { FILTRO } )   // todos los que coinciden
```

mongosh

```
db.usuarios.deleteOne({ email: "juan@mail.com" });

db.usuarios.deleteMany({ ciudad: "Azul" });

// ¡filtro vacío = borra TODO!
db.usuarios.deleteMany({});
```

**BUENA PRÁCTICA**

Antes de un deleteMany, corré un find() con el mismo filtro y revisá qué documentos vas a borrar.

No hay ROLLBACK fuera de una transacción.

# El pipeline de agregación: una línea de montaje




```
db.<coleccion>.aggregate([ { etapa1 }, { etapa2 }, ... ])
```

**Conexión con la clase de optimización:** poner $match lo antes posible es "filtrar antes de cruzar". Reduce los documentos que pasan a las etapas siguientes y, si es la primera etapa, puede usar un índice.

# Agregación: el mismo reporte en MongoDB y en SQL

## "Los 3 vendedores que más notebooks vendieron"

mongosh

```
db.ventas.aggregate([
  { $match: { producto: "notebook" } },
  { $group: {
      _id: "$vendedor",
      totalVentas: { $sum: "$cantidad" }
  } },
  { $sort:  { totalVentas: -1 } },
  { $limit: 3 }
]);
```

SQL

```
SELECT vendedor AS _id,
  SUM(cantidad) AS totalVentas
FROM ventas
WHERE producto = 'notebook'
GROUP BY vendedor
ORDER BY totalVentas DESC
LIMIT 3;
```

*$match ↔ WHERE · $group ↔ GROUP BY + agregación · $sort ↔ ORDER BY · $limit ↔ LIMIT*

# Las etapas más usadas

| Etapa | Qué hace | SQL aproximado |
|---|---|---|
| `$match` | Filtra documentos | `WHERE / HAVING` |
| `$project` | Elige, renombra o calcula campos | `SELECT col, expr AS alias` |
| `$group` | Agrupa y acumula ($sum, $avg, $min, $max, $count) | `GROUP BY` |
| `$sort` | Ordena | `ORDER BY` |
| `$limit / $skip` | Recorta / saltea resultados | `LIMIT / OFFSET` |
| `$unwind` | Genera un documento por cada elemento de un array | `(sin equivalente directo)` |
| `$lookup` | Une con otra colección | `LEFT OUTER JOIN` |
| `$count` | Cuenta los documentos que llegan | `SELECT COUNT(*)` |

# $lookup y $unwind: un JOIN en MongoDB

mongosh

```
db.pedidos.aggregate([
  { $lookup: {
      from: "usuarios",
      localField: "usuario_id",
      foreignField: "_id",
      as: "usuario"
  } },
  { $unwind: "$usuario" },
  { $project: { _id: 0, total: 1,
                cliente: "$usuario.nombre" } }
]);
```

## Paso a paso

1. **$lookup**
   Para cada pedido busca en usuarios el _id igual a usuario_id y lo agrega como array "usuario".
2. **$unwind**
   Convierte ese array de un elemento en un subdocumento.
3. **$project**
   Deja solo total y el nombre del cliente.

**Muchos $lookup = quizás convenía embeber.**

# Índices en MongoDB

mongosh

```
// simple, ascendente (1) o descendente (-1)
db.usuarios.createIndex({ edad: 1 }, { name: "idx_edad" });

// compuesto: filtra por ciudad y ordena por edad
db.usuarios.createIndex({ ciudad: 1, edad: -1 });

// único: no permite dos usuarios con el mismo email
db.usuarios.createIndex({ email: 1 }, { unique: true });

db.usuarios.getIndexes();
db.usuarios.dropIndex("idx_edad");
```

### IGUAL QUE EN SQL

Árboles B: aceleran filtros y orden, pero ocupan espacio y hacen más lentas las escrituras.

### ÍNDICES COMPUESTOS

El orden importa. Regla ESR: igualdad, después orden (sort), al final rango.

# Conectando con la clase anterior: explain()

```
db.usuarios.find({ edad: { $gt: 30 } }).explain("executionStats")
```

**SIN ÍNDICE**

```
winningPlan: {
  stage: "COLLSCAN" }
executionStats: {
  nReturned:            1240,
  totalDocsExamined:    100000,
  executionTimeMillis: 48 }
```

**CON idx_edad**

```
winningPlan: {
  stage: "FETCH",
  inputStage: { stage: "IXSCAN" } }
executionStats: {
  nReturned:            1240,
  totalDocsExamined:    1240,
  executionTimeMillis: 3 }
```

**COLLSCAN ≈ Seq Scan · IXSCAN ≈ Index Scan**

Si totalDocsExamined es mucho mayor que nReturned, falta un índice. (Valores ilustrativos.)

CIERRE

# Ideas para llevarse

1. **NoSQL nace para escalar horizontalmente y flexibilizar el esquema**
   No reemplaza al relacional: amplía las opciones.

2. **CAP: ante una partición se elige entre consistencia y disponibilidad**
   ACID y BASE son dos puntos de partida; muchos motores permiten ajustar.

3. **Cada modelo sirve para un patrón de acceso**
   Documentos, clave-valor, columna ancha y grafos: no hay uno mejor en general.

4. **En documental, se modela a partir de las consultas**
   Embeber lo que se lee junto; referenciar lo que crece sin límite o se comparte.

5. **Los índices y los planes de ejecución también importan en NoSQL**
   explain() cumple el mismo rol que EXPLAIN ANALYZE.

# Bibliografía y documentación


- **Sadalage, P. J.; Fowler, M. — NoSQL Distilled**
  Addison-Wesley, 2012. Introducción breve a modelos de agregados, distribución y consistencia.

- **Kleppmann, M. — Designing Data-Intensive Applications**
  O'Reilly, 2017. Caps. 2 (modelos de datos), 5 (replicación), 6 (particionamiento) y 9 (consistencia).

- **Brewer, E. — CAP Twelve Years Later: How the "Rules" Have Changed**
  IEEE Computer, 45(2), 2012.

- **Documentación oficial de MongoDB**
  https://www.mongodb.com/docs/manual/ — CRUD, agregación, índices, modelado y transacciones.

- **Documentación de Redis, Apache Cassandra y Neo4j**
  redis.io/docs · cassandra.apache.org/doc · neo4j.com/docs/cypher-manual